Day 20 - Functions: is_prime and the Grade Counts Function

Sunday, 8 November 2026 | Swagat (Smooth) | Python: modulo, loops with return, dict counting, tests

1. Key ideas

- **Modulo:** % gives the remainder. number = quotient x divisor + remainder, so 12 % 5 = 2. Remainder 0 means it divides exactly. n % 2 == 0 tests even. 5 % 0 raises ZeroDivisionError. == compares, = assigns.
- **is_prime(n):** n must be greater than 1 and have no divisor from 2 up to its square root. Guard first: if n < 2 return False. Then the loop; return False inside the if (the loop stops at once); return True after the loop (only reached if no divisor was found).
- **Square-root stop:** divisors come in pairs (2 x 18, 3 x 12, 4 x 9, 6 x 6). Both numbers cannot be above the square root (7 x 7 = 49 is already above 36), so one of each pair is at most the square root. Test only d up to int(n ** 0.5) + 1. int() cuts decimals, it does not round, and range() accepts only integers: a float raises TypeError: 'float' object cannot be interpreted as an integer. The + 1 is needed because the stop value is excluded.
- **Empty range:** range(2, 2) is empty, so the loop body never runs and the function falls through to return True. This is right for 2 and 3 and wrong for 0 and 1, which is why the guard exists.
- **Dict counting engine:** work out the key first (here the grade, which is not in the data). Then two separate ifs: first-time step (if grade not in result: result[grade] = 0) and update (result[grade] += 1) which runs for every mark, including the first. in checks keys on a dict and values on a list. A dict keeps insertion order.
- **Tests:** assert raises AssertionError if the claim is False and does nothing if True. Choose tests that each guard against a named bug.

2. Examples (all run and checked)

```
def is_prime(n):
    if n < 2:
        return False
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            return False
    return True

assert is_prime(-5) == False   # guard
assert is_prime(1) == False    # guard
assert is_prime(2) == True     # guard boundary
assert is_prime(4) == False    # needs the + 1
assert is_prime(9) == False    # prime square, needs the + 1
assert is_prime(97) == True    # normal prime
# primes below 30: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

```
def grade_counts(marks):
    result = {}
    for mark in marks:
        if mark >= 75:
            grade = "A"
        elif mark >= 50:
            grade = "B"
        else:
            grade = "C"
        if grade not in result:
            result[grade] = 0
        result[grade] += 1
    return result

grade_counts([85, 42, 67, 91, 30, 75])  # {'A': 3, 'C': 2, 'B': 1}
grade_counts([])                        # {}
grade_counts([75, 74, 50, 49])          # {'A': 1, 'B': 2, 'C': 1}
```

3. My mistakes and the fix

Mistake	Fix
Gave single words for the is_prime blanks; wrote print for the last line and a bare return.	Exact statements: return False inside the if, return True after the loop. print gives the caller None.
Wrote if grade not in marks.	Check the right container: if grade not in result. A letter is never in a list of numbers, so the count reset every time.
Said update (+= 1) happens when the same grade comes again.	The update runs for every mark, including the first. The first-time step only creates the key at 0.
Wrote count += 1.	count is not the dict. Use result[grade] += 1 (count += 1 raises NameError here).
Wrote =< and used return outside a function.	Operators are <= and >=. return outside a function is a SyntaxError.
Chained 50 <= mark <= 74 leaves a gap for 74.5.	elif mark >= 50 covers everything below 75 without gaps.

Mistake	Fix
Said 97 is the largest prime; said an empty range gives False.	There is no largest prime. An empty range skips the loop and gives True.
Said the guard stops 6 and 8 being wrong.	6 and 8 break only without + 1. Without the guard, 0 and 1 give True and negatives raise TypeError.
Reasons were missing: the square-root why, and the result of the dict order.	Divisor pairs show one of each pair is at most the square root; dict order is insertion order.

4. How I improved

- Wrote the full grade_counts function correctly with two separate ifs and the right return placement; output matched on the first run.
- Moved from reading call patterns by guesswork to naming the exact SyntaxError text, and gave the first-time step and update correctly after one correction.
- Reasoned correctly that None beats 0 and that the loop stops at the first return; found the boundary bugs behind the + 1 and the guard with checked examples.

5. Practice to retry (no solutions)

- Write is_even(n) that returns True or False. Test with 0, -3 and 10.
- Write count_divisors(n) that returns how many numbers from 1 to n divide n exactly. Test with 1, 12 and 13.
- Write grade_of(mark) as its own function and use it inside grade_counts. Test with 74.5.
- For n = 25 with the + 1 removed, give the loop range, the d values tested, the result and why it is wrong.
- Count the even and odd numbers in [3, 8, 10, 7, 4] into a dict. Name the key, value, first-time step and update before coding.

6. Recall questions

- What does % give, and how do you test even and odd?
- Why does is_prime stop at the square root, and why + 1 and int()?
- Where does each return sit in is_prime, and what does the empty range do?
- In the dict counting engine, when does the update run, and what does the first-time step do?
- Which test input catches which bug (the guard, the + 1)?

7. Open items

- Redo the first-time step and update from memory on a new shape of problem.
- Split grade_counts into a grade_of helper and the counting loop.
- Carried forward: Exceptions and modules, Files and CSV, README/.gitignore, Git recall and conflict redo, pivot hardest-problem redo, XLOOKUP/INDEX+MATCH recall, Charts/conditional formatting/validation.